

Ryan Stoddard

Mar. 10th, 2026

Foundations of Python Programming

Assignment 07

<https://github.com/RyanStoddard/IntroToProg-Python-Mod07>

Classes and Inheritance

Introduction

In this document I will describe the steps I took to complete Assignment 07. This assignment asks us to create a python program that displays a menu. Depending on the user's choice within the menu they will be allowed to perform different tasks such as inputting data, displaying data, and writing data to a .JSON file. This exercise is nearly identical to assignment 6 except that it introduces constructors, object instances, inheritance, and getter/setter pairs.

The main difference between this week's assignment and last week's is that we are now including a parent class (Person) and a child class (Student). These classes will hold the first name, last name, and course name properties. Which allows us to take advantage of object-oriented features to process the data instead of dealing with dictionaries like we did in the last assignment. The main downside of this approach is that our custom objects don't automatically work well with JSON files. We need to convert the JSON data we read in from the external file into something useable before we can do that file processing. And of course, the opposite is also true, we must convert the data back in order to update the external JSON file.

Creating the Person Class

To create the Person class we start in a similar way as we did with FileProcessor and IO last assignment. You define the class name followed by some descriptive text. But then we call the constructor method (`__init__`) which sets any created object's attributes (Figure 1).

```
41  @
42      def __init__(self, student_first_name: str = "", student_last_name: str = ""):
43          self.student_first_name = student_first_name
44          self.student_last_name = student_last_name
```

Figure 1: Example of the Person Constructor

The syntax is a little confusing, but "self" is a bit like a placeholder that is related to the object's location in memory. Later, when we use this data in the code, "self" will be replaced with the name of the class (example, `student.student_first_name`).

Next we create the getter/setter pair for the properties `student_first_name` and `student_last_name`. The getter is created by using the `@property` decorator. And the setter is created with the `@student_first_name.setter` or `@student_last_name.setter` decorator. Essentially the getter is used anytime that property is called from the code. And the setter is used only when the property is created. This keeps us from accidentally changing an object's property later on in the code. This is also a good

place to put any basic formatting or data validation we want to be enacted on all names. In this case when we “get” the first name property it will always be in title case. And when we set the first name property it checks to make sure there are no numbers in it (Figure 2).

```
45         @property
46         def student_first_name(self):
47             return self.__student_first_name.title()
48
49         @student_first_name.setter
50         def student_first_name(self, value: str):
51             if value.isalpha() or value == "":
52                 self.__student_first_name = value
53             else:
54                 raise ValueError("The first name should not contain numbers.")
55
```

Figure 2: Example of the Getter/Setter Pair

After doing this for both the first name and last name properties we can also do a string override for Person. `__str__` is another one of those “magic methods” like `__init__` that exists automatically. By default it returns its location in memory. But now it returns the property values separated by a comma (Figure 3).

```
67         # String Override for Person
68         def __str__(self):
69             return f'{self.student_first_name},{self.student_last_name}'
70
```

Figure 3: Example of String Override the Person Class

Creating the Student Class

The Student Class is set up in a similar way. But it will be a child of the Person class. This means it will inherit the first name and last name properties. However, will have to add the course name property since it was not part of the Person Class. We use the `super().__init__` method to inherit the first and last names, and use `self.course_name` to add the course name. The rest is the same setting up the Person Class (Figure 4).

```
84         # Constructor, Setter, and Getter
85         def __init__(self, student_first_name: str = "", student_last_name: str = "", course_name: str = ""):
86             super().__init__(student_first_name = student_first_name, student_last_name = student_last_name)
87             self.course_name = course_name
88
89         @property 5 usages (2 dynamic)
90         def course_name(self):
91             return self.__course_name
92
93         @course_name.setter 4 usages (3 dynamic)
94         def course_name(self, value: str):
95             self.__course_name = value
96
97         # String Override for Student
98         def __str__(self):
99             return f"{self.student_first_name},{self.student_last_name},{self.course_name}"
```

Figure 4: Example of the Student Class

Data Conversions in FileProcessor Class Functions

Now that we are using custom objects to contain our properties there are a number of changes we have to make to some of the existing functions. “read_data_from_file” needs to convert the dictionary data into student data. We do this by saving the json.load information into a local variable called list_of_dictionary_data, and then use a for loop to populate a list of student objects using the Student class (Figure 5).

```
125         try:
126             # Get a list of dictionary rows from the data file
127             file = open(file_name, "r")
128             list_of_dictionary_data = json.load(file)
129             for student in list_of_dictionary_data:
130                 student_object: Student = Student(student_first_name=student["FirstName"],
131                                                    student_last_name=student["LastName"],
132                                                    course_name=student["CourseName"])
133                 student_data.append(student_object)
134             file.close()
135         except FileNotFoundError as e:
136             IO.output_error_messages(message: "Text file must exist before running this script!", e)
137         except Exception as e:
138             IO.output_error_messages(message: "There was a non-specific error!", e)
139         finally:
140             if file.closed == False:
141                 file.close()
142         return student_data
```

Figure 5: Example the “read_data_from_file” Function

Similarly, we do the opposite for “write_data_to_file” (Figure 6).

```
159         try:
160             list_of_dictionary_data: list = []
161             for student in student_data: # Convert List of Student objects to list of dictionary rows.
162                 student_json: dict \
163                     = {"FirstName": student.student_first_name,
164                        "LastName": student.student_last_name,
165                        "CourseName": student.course_name}
166                 list_of_dictionary_data.append(student_json)
167
168             file = open(file_name, "w")
169             json.dump(list_of_dictionary_data, file, indent=2)
170             file.close()
171         except TypeError as e:
172             IO.output_error_messages(message: "Please check that the data is a valid JSON format", e)
173         except Exception as e:
174             IO.output_error_messages(message: "There was a non-specific error!", e)
175         finally:
176             if file.closed == False:
177                 file.close()
```

Figure 6: Example the “write_data_to_file” Function

Data Conversions in IO Class Functions

The functions “output_student_and_course_names” and “input_student_data” require similar attention. (Figures 7 and 8)

```
252         print("-" * 50)
253         for student in student_data:
254             message = "{} {} is enrolled in {}".format(student.student_first_name, student.student_last_name, student.course_name)
255             print(message)
256         print("-" * 50)
```

Figure 7: Example the “output_student_courses” Function

```
270         try:
271             student = Student()
272             student.student_first_name = input("Enter the student's first name: ")
273             student.student_last_name = input("Enter the student's last name: ")
274             student.course_name = input("Please enter the name of the course: ")
275             student_data.append(student)
276             message = "You have registered {} {} for {}."
277             print(message.format(student.student_first_name, student.student_last_name, student.course_name))
278
279         except ValueError as e:
280             IO.output_error_messages(message="One of the values was the correct type of data!", error=e)
281         except Exception as e:
282             IO.output_error_messages(message="Error: There was a problem with your entered data.", error=e)
283         return student_data
```

Figure 8: Example the “input_student_data” Function

Summary

The “Classes and Inheritance” assignment introduces constructors, object instances, inheritance, and getter/setter pairs. We learned a little bit about encapsulation. How we can prevent outside code from accidentally changing our data. We learned a little bit about inheritance and how objects can have relationships with each other. I suppose this further enhances modularity/reusability of the code. But it is beginning to be hard to see. Looking back on the last 5 assignments our code has grown longer each week (63 lines, 88 lines, 131 lines, 197 lines, 321 lines). In that time, we have seen some additional functionality added with error validation as well as much better organization and main body code readability. But the core user experience has remained mostly unchanged. Our methods are becoming more refined. Our internal documentation is getting better. Our code is more robust and harder to break. But since the final product remains static, it is becoming difficult to understand why all this complexity is needed.